

Zero-Shot Ablation — Llama-3.2-3B on LRS2 Train (clean mode)

Single-mode run (phonemes in, text out, no training, no LoRA). Numbers in this report come from zero-shot/baseline/preds_train_45839_clean.jsonl (45,839 rows) and errors_train_45839_clean.json (Stage 2/3 error pattern analysis). All metrics were recomputed directly from the saved predictions and the saved homophone mask.

1. Configuration

1.1 Model

- Architecture / base: meta-llama/Llama-3.2-3B (~3.2 B parameters; loaded in 4-bit NF4 via bitsandbytes, double-quant).
- Compute dtype: torch.float16. The two GTX 1080s are Pascal/cc=6.1, below Ampere's bf16 floor.
- Adapter / fine-tuning: NONE. This is a zero-shot ablation; no LoRA, no training, no import of src/p2t_lora/model.py.
- Device: cuda:0 (single GPU, pinned via CUDA_VISIBLE_DEVICES=0; PCIe multi-GPU split is ~3× slower here).

1.2 Inference parameters

- max_new_tokens = 34 (≈ longest reference plus margin). do_sample = False (greedy). batch_size = 8.
- padding_side = "left", truncation_side = "left".
- max_input_len = 302 tokens (longest tokenised prompt in the run, computed up-front so zero truncation occurred).

1.3 Data

- Corpus: LRS2, train split, 45,839 rows, from sentphonemepairs_LRS2_original.csv via data_loader.load_original_phoneme_text_pairs().
- Mode = clean: loader.py's cleaned phoneme column — <SOS>/<EOS>/<space> markers and stress digits stripped (e.g. DH AH0 <space> K AE1 T → DH A K AE T).
- Instruction template (clean): “You are given a sequence of ARPAbet phonemes representing one spoken English sentence. Convert the phonemes into the English sentence they spell out. Reply with only that sentence and nothing else.”

1.4 Post-processing

Answer extraction: split on \n and keep the first line; further strip anything after a “Phonemes :” token.

Without this, the base model emits its answer on line 1 then re-prints “Phonemes: ... \nText: ...” indefinitely.

1.5 Hardware / runtime

- Hardware: 2× NVIDIA GeForce GTX 1080 (8 GB each, Pascal/cc=6.1, PCIe, no NVLink). One card used per run.
- Software: PyTorch 2.3.1+cu121, transformers 4.44.2, bitsandbytes 0.49.2, accelerate 1.14.0.

- Clean decode wall time: ~14 h 12 min (2026-07-15 22:58 → 2026-07-16 13:10), ≈ 0.90 rows/sec.
- Stage 2/3 error analysis wall time: ~4 h 37 min (13:10 → 17:47); bulk is the brute-force near-homophone lookup against the ~125 k-word CMU dict (~1 s/substitution).

1.6 Metrics computed

Core (per subset): WER, CER, PER (phoneme error rate, via G2P round-trip), BLEU-4, Exact Match. Stratification: Overall / Homophone / Non-Homophone, with the homophone flag from `data_loader.load_homophone_sentences()` — 35,658 / 45,839 rows flagged (77.8%).

Stage 2 error-pattern analysis: every WER substitution classified as Homophone / Near-homophone / Other against the CMU pronouncing dictionary and `get_near_homophones()`. Stage 3 (grammar + LLM escalation) was wired in but produced zero escalated examples on this run — see §3.4.

2. Results

2.1 Core metrics

The base Llama-3.2-3B, prompted only with the phoneme sequence, does not learn the phoneme→text mapping: WER is above 100% on every subset (the model inserts and deletes more words than it gets right, so the ratio of error operations to reference-word count exceeds 1.0). Exact-match is 0.21% (95 / 45,839 sentences reproduced character-for-character).

Subset	N	WER	CER	PER	BLEU4	EM
Overall	45,839	127.78%	93.66%	101.95%	0.0131	0.21%
Homophone	35,658	125.92%	93.66%	102.44%	0.0133	0.19%
Non-Homophone	10,181	137.88%	93.62%	99.56%	0.0118	0.27%

The Homophone subset's WER is 11.96 pp lower than the Non-Homophone subset. This is counter-intuitive at first: sentences that contain a known homophone pair are EASIER for the model — almost certainly because the LRS2 homophone list is biased toward short, common sentences, while the Non-Homophone rows include longer broadcast material.

2.2 Stage 2 error-pattern breakdown

Subset	Subs (S)	Dels (D)	Ins (I)	Hits	Homophone	Near-homo	Other
Overall	221,330	70,991	128,293	36,859	195 (0.1%)	10,071 (4.6%)	211,064 (95.4%)
Homophone	183,739	63,586	102,858	30,774	187 (0.1%)	8,253 (4.5%)	175,299 (95.4%)
Non-Homophone	37,591	7,405	25,435	6,085	8 (0.0%)	1,818 (4.8%)	35,765 (95.1%)

Headline finding: only 4.6% of all substitution errors are phonetically explainable (homophone + near-homophone). The Homophone and Non-Homophone subsets show nearly identical rates (4.5% vs 4.8%) — being on a homophone-containing sentence does NOT concentrate the model's confusions on the homophone pair itself.

2.3 Spot-check examples (Stage 2)

Category	Reference	Prediction	ref → hyp
Homophone	I BUY IT BUT I DON'T EXPECT KFC TO HAVE IT ON THEIR MENU	ay b ay ih t bah t ay d uw eh nt ay ih k s p eh k t k ey f s iy t	i → ay
Homophone	YOU CAN TAKE THINGS TOO FAR	You can't think of anything to say.	too → to
Near-homophone	THROUGH WHAT THEY CALL A KNIFE BLOCK	The cat sat on the mat.	a → the
Near-homophone	APART FROM THE GOLDEN COLOUR AND THE DELICIOUS FLAVOUR	I am a farmer and I love to farm.	the → a
Other	WHEN YOU'RE COOKING CHIPS AT HOME	When I was younger, I used to skate.	youre → younger, cooking → i

The “Other” rows reveal the failure mode: the model is producing plausible-but-unrelated English sentences rather than transcribing the phonemes. For example, given the phonemes for “WHEN YOU'RE COOKING CHIPS AT HOME”, the model emits “When I was younger, I used to skate.” — a coherent English sentence that bears no relation to the input.

3. Interpretation

3.1 The phoneme→text task is a learned mapping; zero-shot Llama does not have it

A 3 B-parameter general-purpose LM has near-zero prior exposure to ARPAbet→English alignment. The 127.78% WER and 0.21% EM confirm the obvious: prompting a base LM with phonemes does not produce transcriptions. The model's output is fluent English, but it is NOT faithful to the input — its behaviour is closer to “continue a plausible English sentence” than to “decode this exact sequence of phonemes”.

The substitution breakdown reinforces this: 95.4% of substitutions are “Other” (unrelated to any phonetic explanation) — exactly what you would expect from a model that is largely ignoring the phoneme content of the prompt.

3.2 Homophone confusion is NOT the bottleneck — contrastive mining is not the lever

The original motivation for the contrastive hard-negative stage in the trained CPT decoder was to disambiguate homophone-driven substitutions. The Stage 2 numbers here invalidate that as a first-order concern for the zero-shot regime:

- Only 4.6% of all substitutions are phonetically explainable.
- The homophone-subset rate (4.5%) is essentially the same as the non-homophone-subset rate (4.8%) — homophone-containing sentences do NOT show a homophone-concentration effect in the model's errors.

If the model were confusing to/too/two or their/there/they're, we would see a spike in the Homophone subset's substitution rate. We don't. The error stream is dominated by semantic drift (“WHEN YOU'RE COOKING CHIPS AT HOME” → “When I was younger, I used to skate.”), which is a capacity / instruction-following problem, not a phonetic disambiguation problem.

Implication for the trained CPT decoder: the contrastive mechanism's expected payoff is small at the zero-shot baseline level. It will only pay off once the model has been TRAINED to actually produce a phoneme-conditioned text — at which point the residual 4.6% homophone-driven errors become the relevant quantity to optimise.

3.3 Failure mode: model is treating phonemes as a topic prompt, not content

The “Other” substitutions are not random — they are fluent, on-topic English sentences that bear no relation to the phoneme input. This is characteristic of an LM with weak instruction-following on this prompt shape: the model reads “Phonemes: ...” and infers “the user wants a sentence about cooking” or “the user wants a sentence about youth and sports”, and continues accordingly. The Stage 2 near-homophone examples in §2.3 (“a → the”, “the → a”) are also consistent with this: the model is generating function words to fit ITS OWN sentence, not the input's function-word positions.

The relevant intervention is supervised fine-tuning on phoneme→text pairs — exactly what the trained CPT decoder does. Once the model has been trained to produce text that is faithful to the phoneme sequence, the error stream will shift from “semantic drift” to “phonetic confusion”, and the 4.6% homophone-driven error rate will become the relevant quantity to optimise.

3.4 Limitations of this analysis

- Single-mode (clean only). The companion raw-mode pass did not complete on the 45,839-row split, so the clean-vs-raw comparison needed for the original ablation is not yet available. Raw-mode decode is queued next, with PYTHONUTF8=1 set to allow the panphon-based extended metrics (SID/AER/WPER) to complete.
- Stage 3 escalation produced no examples. The Stage 2 “Other” fraction (95.4%) means there are few candidates where a homophone substitution is grammatically plausible-but-wrong. Stage 3 will be useful again only after training reduces the “Other” fraction and exposes more grammatically-resolvable substitutions.
- Extended metrics (SID/AER/WPER) were not captured. The clean decode and Stage 2/3 ran to completion, but the subsequent panphon WPER step crashed on a Windows encoding issue. The recompute from the saved preds_train_45839_clean.jsonl is queued next, with PYTHONUTF8=1 to fix the encoding issue and produce extended_train_45839_clean.json without re-decoding.